

IIT Madras Machine Learning Practice (MLP) Project

Technical Report: Heavy Equipment Price Prediction via Multi-Seed GBDT Ensembling

Author: Ramrup Satpati

Institution: IIT Madras BS Degree in Data Science & Applications

Course: Machine Learning Practice (MLP) Project

Final Score: 90.00% | **Grade:** S (10.0 GPA)

License: (C-) Copyleft 2026 Ramrup Satpati | All Rights Reversed (GNU GPLv3)

Abstract

Accurate price forecasting for heavy industrial machinery (excavators, dozers, wheel loaders) presents unique challenges due to non-linear physical depreciation, usage intensity fluctuations, high-cardinality categorical descriptions, and right-skewed target distributions. This report documents the end-to-end Machine Learning pipeline developed for the Heavy Equipment Price Prediction benchmark. We introduce a regex-based physical specification parser, domain-specific temporal depreciation ratios, out-of-fold frequency encoding, and a 5-seed weighted log-space GBDT ensemble combining Deep XGBoost (45%), LightGBM (35%), and CatBoost (20%). Furthermore, we derive and apply an empirical Jensen's Inequality post-processing correction multiplier (1.001300) to eliminate log-transformation downward variance bias. The resulting architecture achieves a validation Root Mean Squared Logarithmic Error (RMSLE) of **0.18663**, placing in the top percentile of competition benchmarks.

1. Introduction & Problem Formulation

In the commercial machinery market, equipment valuation depends on physical wear, operational age, equipment size class, and macro-economic sale timing. Standard regression models fail when applied directly to raw transaction records due to unformatted text descriptions, missing telemetry data, and exponential price decay.

The evaluation metric specified for this benchmark is **Root Mean Squared Logarithmic Error (RMSLE)**:

$$\text{RMSLE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\log(y_i + 1) - \log(\hat{y}_i + 1))^2}$$

To directly minimize RMSLE, all models are trained using log-transformed target values:

$$z_i = \log(y_i + 1)$$

2. Feature Engineering & Specification Extraction

Raw dataset columns contain rich but unstructured textual descriptors. We designed modular regex parsers in `src/feature_engineering.py` to extract continuous physical telemetry:

1. **Horsepower (HP):** Parsed via regex `(\d+(?:\.\d+)?)\s*(?:hp|horsepower)` to capture engine capacity.
 2. **Operating Tonnage:** Parsed via regex `(\d+(?:\.\d+)?)\s*(?:ton|tonnage)` to extract machine mass.
 3. **Bucket Yardage:** Parsed via regex `(\d+(?:\.\d+)?)\s*(?:yd|yard)` for excavator bucket capacity.
 4. **Temporal Features:** Sale Year, Machine Age ($\text{SaleYear} - \text{YearMade}$), and Operating Hours per Year ($\frac{\text{MeterHours}}{\text{MachineAge}+1}$).
 5. **Depreciation Interaction Ratios:** Log-transformed age ratios ($\log(\text{MachineAge}+1)$) and usage intensity metrics.
-

3. Categorical Preprocessing & Encoding

To prevent high-cardinality categorical variables (e.g., `ModelID`, `fiModelDesc`, `fiBaseModel`, `ProductGroup`, `Enclosure`) from causing dimensionality explosion:

- **Frequency Encoding:** Categorical values are mapped to their normalized empirical frequency within the training split.
 - **Native Categorical Dtypes:** Categorical columns are converted to pandas category dtypes, allowing LightGBM and CatBoost to perform optimal histogram binning and ordered target encoding without dummy variable expansion.
-

4. Multi-Model Gradient Boosting Architecture

We constructed three distinct gradient boosted decision tree (GBDT) model families in `src/models.py`:

1. **Deep XGBoost Regressor (Weight: 45%):**
 - `max_depth: 9, learning_rate: 0.015, n_estimators: 2500, colsample_bytree: 0.75, subsample: 0.85, tree_method: "hist".`
2. **Leaf-Wise LightGBM Regressor (Weight: 35%):**
 - `num_leaves: 127, learning_rate: 0.012, n_estimators: 3000, subsample: 0.80, colsample_bytree: 0.70.`
3. **Ordered CatBoost Regressor (Weight: 20%):**
 - `depth: 8, learning_rate: 0.02, iterations: 2500, l2_leaf_reg: 3.0.`

Each model architecture was trained across **5 random seeds** (42, 100, 2026, 777, 999) to stabilize prediction variance across cross-validation folds.

5. Jensen’s Inequality Bias Correction & Blending

When predicting log-transformed target variables $\hat{z}_i$, converting back to currency units via exponentiation ($\hat{y}_i = e^{\hat{z}_i} - 1$) introduces a systematic downward bias due to **Jensen’s Inequality**:

$$\mathbb{E}[e^Z] \geq e^{\mathbb{E}[Z]}$$

To correct for variance loss under log-averaging, we apply an empirical post-processing multiplier:

$$\hat{y}_{\text{final}} = (\text{expm1}(\text{LogBlend})) \times 1.001300$$

Where $\text{LogBlend} = 0.45 \cdot \hat{z}_{\text{XGB}} + 0.35 \cdot \hat{z}_{\text{LGB}} + 0.20 \cdot \hat{z}_{\text{CAT}}$.

6. Empirical Results & Validation Benchmark

Model / Pipeline Stage	Validation RMSLE	CV Std Dev	Improvement vs Baseline
Baseline	0.24510	± 0.0042	Baseline
Random Forest Single	0.19840	± 0.0028	+19.05%
LightGBM + Basic Ratios			
Deep XGBoost (Depth = 9)	0.19120	± 0.0021	+21.99%
5-Seed Log-Blend + Jensen Multiplier	0.18663	± 0.0009	+23.85%

7. Reproducibility & Source Modules

The project is structured according to production software engineering standards:

- `src/feature_engineering.py`: Regex physical parsers and temporal features.
- `src/preprocessing.py`: Frequency encoding and median missing value imputation.
- `src/models.py`: Hyperparameter configurations for XGBoost, LightGBM, CatBoost.

- `src/evaluate.py`: RMSLE metrics, log-space ensembling, and Jensen multiplier correction.
 - `notebooks/`: 5 sequential milestone notebooks documenting EDA, spec parsing, feature engineering, multi-model tuning, and ensembling.
-

8. Conclusion & License

The multi-seed GBDT ensemble pipeline combined with domain-specific regex spec parsing and Jensen's inequality multiplier achieves exceptional predictive performance (0.18663 RMSLE), securing an **S-Grade (90.00%)** in the IIT Madras MLP Project.

(C-) **Copyright 2026 Ramrup Satpati | All Rights Reserved**

Released under the **GNU General Public License v3.0 (GPLv3)**.